

CKA - Scheduling 섹션 핵심 정리

1. Scheduling 개요

스케줄러가 하는 일은 새로 생긴 Pod를 어느 노드에 배치할지 결정하는 것.

- kube-scheduler 가 기본 스케줄러
- 노드의 자원 상태, taint, affinity, 우선순위 등을 종합해서 가장 적합한 노드 선택
- Pod yaml에 nodeName 이 비어있으면 스케줄러가 채워줌

2. Manual Scheduling

스케줄러 없이 사람이 직접 노드를 지정하는 방법.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  nodeName: node1  # 스케줄러 거치지 않고 바로 이 노드로
  containers:
  - name: nginx
    image: nginx
```

이미 떠 있는 Pod는 nodeName 을 수정할 수 없음 (불변 필드) → 삭제 후 재생성하거나 Binding 오브젝트를 직접 만들어야 함.

3. Labels and Selectors

쿠버네티스 리소스에 붙이는 검색/식별용 태그.

```
metadata:
  labels:
    app: my-app  # ① 이 오브젝트(Deployment) 자체에 붙는 태그

spec:
  template:
    metadata:
```

```
labels:      # ② 이 오브젝트가 만드는 Pod에 붙는 태그
  app: my-app
```

	metadata.labels	template.metadata.labels
붙는 대상	리소스 자체 (Deployment 등)	생성되는 Pod
조회 명령	<code>kubectl get deploy -l app=my-app</code>	<code>kubectl get pod -l app=my-app</code>
Service selector 연동	무관	연동됨

ReplicaSet 핵심 규칙: `selector.matchLabels` 와 `template.metadata.labels` 는 반드시 일치해야 함. 다른 ReplicaSet이 자기가 만든 Pod를 인식 못 함.

```
spec:
  selector:
    matchLabels:
      tier: front-end  # "나 이 태그 가진 Pod 관리할게" 선언
  template:
    metadata:
      labels:
        tier: front-end  # 실제 Pod한테 붙는 태그 (반드시 selector와 동일)
```

구조 고정값: `selector` 밑에는 항상 `matchLabels`, `template` 밑에는 `metadata.labels`.

4. Taints and Tolerations

- **Taint** = 노드에 붙이는 "함부로 오지마" 딱지 (노드 → Pod 방향, 거부)
- **Toleration** = Pod에 붙이는 "그 딱지 괜찮아" 허용 (해당 taint를 견딘다는 뜻)

한국어 의미: taint(오염) ↔ toleration(내성). 독을 뿌려놓고(taint), 해독제 가진 Pod(toleration)만 들어올 수 있는 비유.

명령어 (master 노드에서 실행)

```
# taint 걸기
kubectl taint node node1 app=blue:NoSchedule

# taint 제거 (뒤에 - 붙임)
kubectl taint node node1 app=blue:NoSchedule-
```

Effect 3종류

Effect	새 Pod	기존 Pod
NoSchedule	못 옴	유지
PreferNoSchedule	가능하면 오지마 (강제 아님)	유지
NoExecute	못 옴	쫓겨남

Pod의 toleration 설정 (yaml)

```
tolerations:
- key: "app"
  operator: "Equal"
  value: "blue"
  effect: "NoSchedule"
```

operator 차이

- Equal → key, value, effect **셋 다** 일치해야 통과
- Exists → key, effect **만** 일치하면 통과 (value는 적어도 무시됨)

자동 NoExecute 사례: 노드가 다운되면 쿠버네티스가 자동으로 해당 노드에 NoExecute taint를 걸어서, toleration 없는 기존 Pod들을 다른 정상 노드로 옮김 (자동 장애 복구).

중요: master 노드에는 기본적으로 NoSchedule taint가 걸려 있어서 일반 Pod가 못 올라감.

```
kubectl describe node master | grep Taint
```

5. Node Selectors

Pod가 **직접 노드를 선택**하는 가장 단순한 방법 (Taint/Toleration과 반대 방향: Pod → 노드).

```
# 1단계: 노드에 label 붙이기
kubectl label node node1 size=large
```

```
# 2단계: Pod yaml에서 지정
spec:
  nodeSelector:
    size: large
```

한계: OR/NOT 같은 복잡한 조건 불가 (예: "large 또는 medium", "small 제외" 불가능).

6. Node Affinity

Node Selector의 상위 호환. 복잡한 조건(OR, NOT) 가능.

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: size
                operator: In          # In, NotIn, Exists, DoesNotExist
                values:
                  - large
                  - medium
```

operator 종류

- In → 값들 중 하나면 OK (OR)
- NotIn → 해당 값 제외 (NOT)
- Exists → 키만 있으면 OK
- DoesNotExist → 해당 키 없는 노드만

Type 구분 (DuringScheduling / DuringExecution)

	DuringScheduling (배치 시)	DuringExecution (실행 중)
required	반드시 지켜야 함	무시 (Ignored)
preferred	가능하면 지킴, 못 찾으면 다른 노드라도 배치	무시 (Ignored)

- 현재는 RequiredDuringExecution 타입은 없음 → 즉, 노드 라벨이 나중에 바뀌어도 이미 실행 중인 Pod는 쫓겨나지 않음.
- CKA는 오픈북 시험이므로 이 긴 필드명을 외우기보다 **구조와 의미**를 이해하고 공식 문서에서 찾을 수 있으면 충분.

7. Taints & Tolerations vs Node Affinity

	주체	방향	한계
Taints/ Tolerations	노드가 Pod를 거부/허용	노드 → Pod	다른 Pod가 그 노드로 오는 건 못 막음 (toleration 있으면 어디든 갈 수 있음)
Node Affinity	Pod가 노드를 선택	Pod → 노드	다른 Pod가 같은 노드로 오는 건 못 막음

완벽한 격리가 필요하면 둘을 함께 사용:

Taint → 다른 Pod들이 이 노드에 못 오게 막음
Affinity → 내 Pod는 반드시 이 노드로 가게 보장

→ "이 노드엔 이 Pod만" 보장 가능.

8. Resource Requirements

Pod한테 CPU/메모리를 얼마나 줄지 정의.

```
resources:
  requests:
    memory: "1Gi"
    cpu: "1"
  limits:
    memory: "2Gi"
    cpu: "2"
```

	requests	limits
의미	최소 보장량	최대 허용량
스케줄링 영향	노드 자리 계산 기준	영향 없음

requests의 역할: 노드 전체의 메모리 과부하를 막는 안전장치. requests를 제대로 안 잡으면 노드 자체가 메모리 부족(Eviction)에 빠질 수 있음.

단위

CPU: "1" (1코어), "500m" (0.5코어, m=milli=1/1000), "0.5" (동일)
메모리: Ki/Mi/Gi (2진법, 1024 단위) vs K/M/G (10진법, 1000 단위)
→ 1Gi ≈ 1.07G (Gi가 살짝 더 큼)

9. Resource Limits

limits를 초과했을 때의 동작 차이가 핵심.

자원	초과 시 동작
CPU	throttling (속도 제한만, 안 죽음)

자원	초과 시 동작
메모리	OOMKilled (강제 종료)

OOMKilled = Out Of Memory Killed

```
kubectl describe pod 파드이름
# Last State: Terminated, Reason: OOMKilled
```

트러블슈팅 시나리오: Pod가 계속 재시작됨 → describe로 OOMKilled 확인 → limits.memory 부족이 원인 이면 늘려서 해결. (단, 메모리 누수 버그면 늘려도 또 죽으므로 구분 필요)

노드 차원 문제와 구분

문제	원인	해결
Pod만 죽음 (OOMKilled)	Pod의 limits가 너무 작음	limits 늘리기
노드 전체 과부하 (Evicted)	requests 미설정으로 여러 Pod가 자원 과사용	requests 제대로 설정

10. DaemonSets

모든 노드에 Pod를 정확히 1개씩 자동 배치하는 오브젝트.

- Deployment처럼 replicas 개수를 정하는 게 아니라 노드 개수만큼 자동 생성
- 새 노드가 추가되면 자동으로 그 노드에도 Pod 생성

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: monitoring-agent
spec:
  selector:
    matchLabels:
      app: monitoring
  template:
    metadata:
      labels:
        app: monitoring
    spec:
      containers:
        - name: monitoring-agent
          image: monitoring-agent
```

용도: 로그 수집기, 모니터링 에이전트, 네트워크 플러그인(kube-proxy 등) — 모든 노드에 똑같이 떠 있어야 하는 것들.

kube-proxy: Service ↔ Pod 사이 트래픽 라우팅 담당. Pod IP가 계속 바뀌므로 각 노드마다 트래픽 규칙을 관리하는 kube-proxy가 DaemonSet으로 배포됨.

```
kubectl get ds -n kube-system # kube-proxy 확인 가능
```

11. Static Pods

kube-apiserver 없이 **kubelet**이 직접 관리하는 Pod.

- 특정 노드의 `/etc/kubernetes/manifests` 디렉터리에 yaml 파일을 두면 kubelet이 알아서 해당 Pod를 생성/유지
- 컨트롤 플레인 컴포넌트(`kube-apiserver` , `etcd` , `kube-scheduler` , `kube-controller-manager`) 자체가 보통 static pod로 떠 있음 (닭이 먼저냐 달걀이 먼저냐 문제 해결: apiserver 뜨기 전에 누가 apiserver를 띄우나 → kubelet이 직접)
- 이름 끝에 노드 이름이 자동으로 붙음 (`etcd-controlplane` 같은 식)
- `kubectl describe pod` 로 보이긴 하지만 `kubectl delete` 로 못 지움 (manifest 파일을 지워야 사라짐)

```
# kubelet 설정에서 static pod 경로 확인
ps -ef | grep kubelet | grep pod-manifest-path
```

12. Priority Classes

Pod에 **우선순위**를 매겨서, 자원이 부족할 때 어떤 Pod를 먼저 스케줄링하고 어떤 Pod를 쫓아낼지 결정.

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
preemptionPolicy: PreemptLowerPriority # preemption 동작 방식 지정
description: "중요한 서비스용"
```

```
# Pod에서 사용
spec:
  priorityClassName: high-priority
```

- 숫자가 클수록 우선순위 높음
- `globalDefault: true` 로 설정한 `PriorityClass`가 하나 있으면, `priorityClassName` 안 정한 Pod들이 자동으로 그 클래스를 따름

Priority 값 범위

일반 Pod → 0 ~ 1,000,000,000
시스템 컴포넌트용 → 1,000,000,000 이상 (`system-cluster-critical`, `system-node-critical`)

쿠버네티스가 기본으로 제공하는 시스템용 `PriorityClass`:

```
kubectl get priorityclass
# system-cluster-critical 2000000000
# system-node-critical   2000001000
```

Preemption (선점)

자원이 부족해서 Pending 상태인 높은 우선순위 Pod를 배치하기 위해 낮은 우선순위 Pod를 강제로 쫓아내는 것.

흐름

높은 우선순위 Pod 생성 → 노드에 자리 없음 → Pending
→ 스케줄러가 낮은 우선순위 Pod 쫓아냄
→ 확보된 자리에 높은 우선순위 Pod 배치

preemptionPolicy 옵션 2가지

옵션	동작
<code>PreemptLowerPriority</code>	기본값. 자원 부족 시 낮은 우선순위 Pod를 쫓아내고 그 자리에 들어감
<code>Never</code>	우선순위는 높지만 preemption 하지 않음. 자리가 날 때까지 그냥 기다림

```
# preemption 하지 않는 PriorityClass 예시
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
```



```
name: high-priority-no-preemption
value: 1000000
preemptionPolicy: Never
globalDefault: false
```

Never를 쓰는 경우: 우선순위는 높게 잡고 싶지만, 실행 중인 다른 Pod를 강제로 죽이는 건 피하고 싶을 때.
(예: 배치 작업처럼 급하지 않지만 자원 경쟁에서 밀리면 안 되는 워크로드)

13. Multiple Schedulers

기본 `kube-scheduler` 외에 **커스텀 스케줄러를 추가로 운영** 가능.

특정 워크로드에 맞는 배치 로직이 필요할 때 (예: GPU 작업 전용 스케줄러) 별도 스케줄러를 만들어 둘 다 운영.

Pod에서 사용할 스케줄러 지정

```
spec:
  schedulerName: my-custom-scheduler
  containers:
  - name: nginx
    image: nginx
```

방식 1: Static Pod로 배포 (강의/CKA 기준)

기본 `kube-scheduler`도 static pod로 떠 있으므로, 커스텀 스케줄러도 같은 방식으로 배포.
파일을 `/etc/kubernetes/manifests/`에 넣으면 kubelet이 자동으로 띄워줌.

파일이 2개 필요

① `my-scheduler-config.yaml` (스케줄러 설정)

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler
leaderElection:
  leaderElect: true
  resourceNamespace: kube-system
  resourceName: lock-object-my-scheduler
```

- `schedulerName` : Pod의 `spec.schedulerName` 과 반드시 일치해야 함

- `leaderElect: true` : 마스터 노드가 여러 대인 HA 환경에서 스케줄러도 여러 개 뜰 수 있으므로, 딱 하나만 실제로 스케줄링하도록 리더를 선출. 마스터가 1대면 `false` 로 해도 됨.

② my-custom-scheduler.yaml (스케줄러 Pod 정의)

```
apiVersion: v1
kind: Pod
metadata:
  name: my-custom-scheduler
  namespace: kube-system
spec:
  containers:
    - name: kube-scheduler
      image: k8s.gcr.io/kube-scheduler:v1.26.0
      command:
        - kube-scheduler
        - --address=127.0.0.1
        - --kubeconfig=/etc/kubernetes/scheduler.conf
        - --config=/etc/kubernetes/my-scheduler-config.yaml
```

방식 2: Deployment로 배포

클러스터 운영 중에 동적으로 추가하거나 replica를 조정할 수 있어 유연함.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-custom-scheduler
  namespace: kube-system
spec:
  replicas: 1
  selector:
    matchLabels:
      component: my-custom-scheduler
  template:
    metadata:
      labels:
        component: my-custom-scheduler
    spec:
      serviceAccountName: my-scheduler
      containers:
        - name: kube-scheduler
```

```
image: k8s.gcr.io/kube-scheduler:v1.26.0
command:
- kube-scheduler
- --config=/etc/kubernetes/my-scheduler-config.yaml
```

두 방식 비교

	Static Pod	Deployment
배포 방식	/etc/kubernetes/manifests/ 에 파일 넣기	kubectl apply -f
관리 주체	kubelet 직접 관리	ReplicaSet이 관리
CKA 시험 기준	강의에서 다루는 방식	실무에서도 많이 사용
공통점	둘 다 my-scheduler-config.yaml 필요	

확인 명령어

```
kubectl get pods -n kube-system | grep scheduler
kubectl get events -o wide # 어떤 스케줄러가 Pod 배치했는지 확인
```

14. Configuring Scheduler Profiles

쿠버네티스 1.18+부터는 스케줄러 여러 개를 따로 안 만들고, **하나의 스케줄러 안에서 여러 프로필 (Profile)**을 설정해서 워크로드별로 다른 스케줄링 전략 적용 가능.

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: default-scheduler
- schedulerName: high-priority-scheduler
plugins:
  score:
    disabled:
      - name: NodeResourcesFit
```

- Extension point(queueSort , filter , score , bind 등) 단위로 plugin을 enable/disable 가능
- Pod에서는 동일하게 schedulerName 으로 프로필 선택

15. Admission Controllers

요청이 인증(Authentication) → 인가(Authorization) 를 통과한 뒤, etcd에 저장되기 직전에 가로채서 검사/수정하는 단계.

사용자 요청

- Authentication (누구나)
- Authorization (권한 있냐)
- Admission Controller (이 요청 내용이 적절한가, 수정 필요한가)
- etcd 저장

- 예: NamespaceExists (없는 네임스페이스에 생성 시도 막기), AlwaysPullImages, DefaultStorageClass 등
- 활성화된 컨트롤러 확인:

```
kube-apiserver -h | grep enable-admission-plugins
```

- kube-apiserver의 static pod yaml에서 --enable-admission-plugins, --disable-admission-plugins 옵션으로 제어

16. Validating and Mutating Admission Controllers

Admission Controller는 두 종류로 나뉨.

종류	역할	예시
Mutating	요청 내용을 수정	누락된 필드에 기본값 채우기, 사이드카 자동 주입
Validating	요청이 규칙에 맞는지 검사만, 통과/거부 결정	특정 라벨 없으면 생성 거부

처리 순서: Mutating이 먼저 실행되고 그 다음 Validating이 실행됨 (수정 끝난 최종 상태를 검증해야 하므로).

요청 → Mutating Admission Controller (수정) → Validating Admission Controller (검증) → etcd

커스텀 Admission Controller: MutatingAdmissionWebhook, ValidatingAdmissionWebhook 을 통해 사용자가 직접 만든 외부 서버(웹훅)를 호출해서 자체 로직으로 검사/수정 가능. 이것 만들려면:

1. Admission Webhook 서버 작성 (요청 받아서 allow/deny 또는 patch 반환)
2. MutatingWebhookConfiguration 또는 ValidatingWebhookConfiguration 오브젝트로 등록

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
```

```
webhooks:
- name: my-webhook.example.com
  clientConfig:
    url: "https://my-webhook-server/validate"
  rules:
  - apiGroups: [""]
    resources: ["pods"]
    operations: ["CREATE"]
```

CKA 시험 팁 요약

- **시험은 오픈북** — kubernetes.io/docs 검색 가능. 긴 yaml 구조는 외우기보다 "이런 게 있었지" 정도로 기억하고 문서에서 빠르게 찾는 연습이 더 중요.
- **트러블슈팅** 문제는 항상 `kubectl describe`, `kubectl get events`, `kubectl logs` 부터 찍어서 에러 메시지/상태를 먼저 확인.
- `kubectl` [동사] [대상] [옵션] 패턴으로 명령어를 외우지 말고 유추하는 습관.